

SOFTWARE ENGINEERING PROJECT REPORT

Vibecheck Cinema

Media Intelligence Dashboard with Movie Data Caching and
Sentiment Analysis

**Discover movies.
Analyse audience opinion.
Reduce repeated API calls.**

University: Thapar University
Course: Software Engineering
Faculty: Mr. Hardik Gujral
Project type: Full-stack web application
Frontend shell: React 19, TypeScript, Vite
Cinema dashboard: HTML, CSS, and JavaScript
Application gateway: Node.js/Express with tRPC integration
Analysis service: FastAPI, SQLAlchemy, Supabase PostgreSQL
External data: TMDB and YouTube APIs
Authentication: Firebase Authentication

Acknowledgement

This project is submitted for the Software Engineering course at Thapar University under the guidance of Mr. Hardik Gujral.

Contents

1	Abstract	1
2	Introduction	1
2.1	Background	1
2.2	Problem Statement	1
2.3	Objectives	1
3	System Overview	2
4	Technology Stack	3
5	Functional Requirements and Features	3
5.1	Authentication and Account Management	3
5.2	Movie Discovery Dashboard	4
5.3	Search, Details, and Analysis	4
5.4	Watchlist and History	4
5.5	Responsive User Experience	5
6	Detailed Architecture	5
6.1	React Authentication Shell	5
6.2	Node/Express Layer	5
6.3	FastAPI Cache Service	6
6.4	Dynamic Expiration	7
6.4.1	Expiry and Fetch Formula	7
7	Data Flow	9
8	Sentiment-Analysis Method	9
9	Database and Persistence Design	10
10	Security, Reliability, and Error Handling	11
11	Testing and Build Strategy	11
12	Evaluation	11
13	Future Enhancements	12
14	Conclusion	12
A	Representative API Endpoints	12
B	Source-Based Notes	13

1 Abstract

Vibecheck Cinema is a web-based media-intelligence platform for discovering films and producing a concise, data-supported view of audience reception. The submitted source contains a React and TypeScript authentication shell, a separate static HTML/CSS/-JavaScript cinema dashboard, a Node.js/Express application server with tRPC support and cinema routes, and a separate FastAPI caching service backed by PostgreSQL. Movie lists and metadata are obtained from The Movie Database (TMDB), while review comments are collected from YouTube and classified using a lightweight lexicon-based sentiment-analysis module.

The principal engineering contribution is the caching layer. Requests are assigned stable cache keys, stored in a PostgreSQL table, and served from the database while their time-to-live (TTL) remains valid. The service applies different and dynamic TTL policies to lists, searches, movie details, and review analysis. This reduces repeated calls to external APIs, improves response time for repeated requests, and provides a single shared result for multiple users. The resulting application supports authentication, movie search, popular and top-rated discovery, trending views, watchlists, local watch history, movie detail pages, review sentiment summaries, profile management, and account lifecycle operations.

2 Introduction

2.1 Background

Movie discovery applications commonly depend on third-party APIs for titles, posters, ratings, trailers, and audience discussion. Directly requesting the same information for every visitor increases latency and consumes external API quotas. It also makes the user experience sensitive to temporary upstream failures. A persistent cache can address these issues by storing normalized responses and reusing them until they should be refreshed.

2.2 Problem Statement

The project addresses three related problems:

1. Users need a single interface for discovering films across several categories rather than separate pages for each data source.
2. Audience reviews are unstructured and difficult to summarize quickly during movie selection.
3. Repeated requests for identical movie data should not repeatedly invoke TMDB and YouTube APIs.

2.3 Objectives

The project objectives inferred from the implementation are:

- provide a polished dashboard for movie discovery and analysis;
- support secure account creation, login, verification, password recovery, Google sign-in, logout, and account deletion;

- retrieve and normalize movie data from TMDB;
- retrieve review comments from YouTube and classify positive and negative statements;
- cache external responses in PostgreSQL with endpoint-specific expiration rules;
- offer watchlist and watch-history features with responsive desktop and mobile navigation; and
- expose health and API endpoints that make deployment and troubleshooting practical.

3 System Overview

The project is organized as a web application plus a specialized data service. The React/-TypeScript code implements the authentication shell and mounts the cinema experience after login. The main cinema interface itself is a static page at `client/public/cinema/app.html`, with behavior in JavaScript and styling in CSS. The Node/Express server serves the application, registers the cinema HTTP routes, and mounts tRPC at `/api/trpc`. The archive also contains a FastAPI service intended to provide persistent PostgreSQL-backed caching. Thus, the project contains two backend layers: the active Node/Express gateway and a separate Python cache-service implementation.

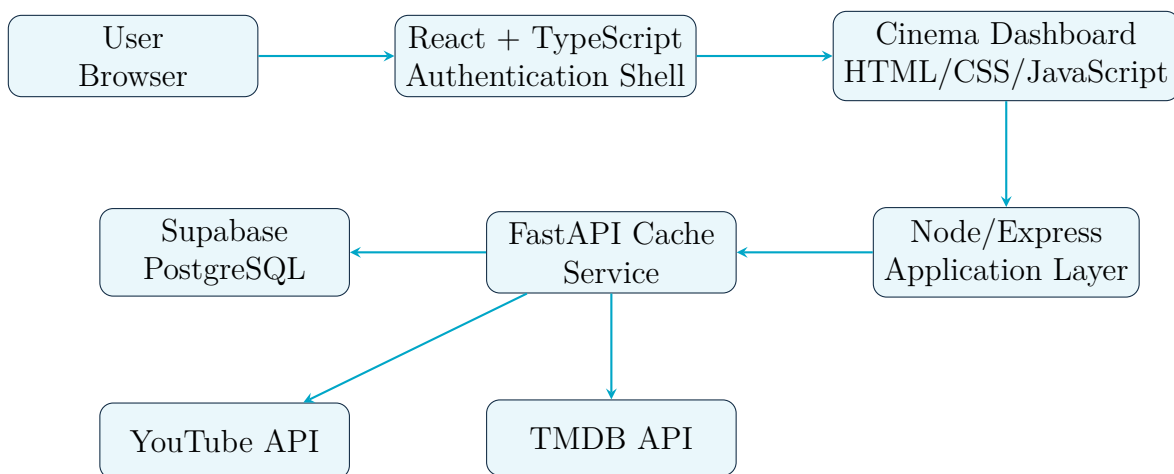


Figure 1: Logical architecture of the Vibecheck Cinema system.

4 Technology Stack

Table 1: Main technologies identified in the source archive.

Layer	Technology and role
Frontend shell	React 19, TypeScript, Vite, Wouter, Tailwind CSS, Radix UI components, Framer Motion, Recharts; used for authentication and the application shell.
Cinema dashboard	Static HTML, CSS, and JavaScript page loaded through <code>/cinema/app.html</code> ; it implements the main movie-discovery interface.
Authentication	Firebase Authentication with email/password, verification, password reset, Google sign-in, reauthentication, and account deletion
Application gateway	Node.js, Express, tRPC, TypeScript, cookie/session and storage helpers; the submitted server registers the cinema routes and the <code>/api/trpc</code> middleware.
Cache service	Python 3, FastAPI, asynchronous HTTP clients, SQLAlchemy async sessions; this is a separate service intended to provide persistent movie-response caching.
Persistence	Supabase PostgreSQL table for cached responses; browser localStorage for watchlist and history
External APIs	TMDB for movie metadata and YouTube Data API for review videos/comments
Testing/build	TypeScript compiler, Vitest, Vite build, esbuild, Prettier, pnpm scripts

5 Functional Requirements and Features

5.1 Authentication and Account Management

The login page supports two modes: login and account creation. New accounts receive an email verification message before access to the cinema dashboard is granted. Existing users can authenticate with email and password or with Google. The implementation also provides password recovery through Firebase, verification-link processing, verification-email resend with cooldown handling, and user feedback through status messages.

The application contains defensive account controls. Failed email/password attempts are counted locally in the page state, and five failed attempts trigger a temporary 60-second lockout. Account deletion requires confirmation and handles Firebase’s recent-login requirement by reauthenticating with either a password credential or Google popup. These flows are integrated with the top-level authentication listener so that the interface changes automatically when the Firebase session changes.

5.2 Movie Discovery Dashboard

The cinema page presents a dark, responsive media dashboard called Vibecheck Cinema. Its navigation includes Dashboard, Watchlist, Trending, and Data Hub views. The dashboard displays popular titles, all-time top-rated titles, currently playing titles, search results, and a watchlist preview. Skeleton placeholders are rendered while asynchronous requests are pending, and a retry block is presented for API or network failures.

5.3 Search, Details, and Analysis

Users can search by movie title from desktop or mobile search fields. Movie cards show poster imagery, release year, rating, and a watchlist control. Selecting a movie opens a detail workflow containing normalized metadata, genre information, runtime, director, cast, trailer information, and the audience-analysis result returned by the cache service.

The review workflow searches YouTube for relevant movie-review videos, collects top-level comments, and classifies sentences using positive and negative lexicons. The interface can therefore show an overall verdict, positive percentage, review counts, aspect-level pros and cons, and representative review sentences.

5.4 Watchlist and History

The watchlist is stored in browser localStorage under the key `vibecheck_watchlist`. It can be toggled from any movie card, is reflected in navigation badges, and can be cleared with confirmation. The application also stores a bounded local history under `vibecheck_history`; each entry includes the movie identifier, title, poster, genres, timestamp, and optional positive-sentiment percentage. The history is limited to the latest 25 analyzed titles.

The source makes the storage choice explicit. The watchlist is read from and written to browser storage, and therefore is not synchronized through Supabase or another server database in the current implementation.

Listing 1: Browser-local watchlist persistence from `client/public/cinema/js/app.js`.

```
const WATCHLIST_KEY = 'vibecheck_watchlist';
const HISTORY_KEY = 'vibecheck_history';

function getWatchlist() {
  try {
    return JSON.parse(localStorage.getItem(WATCHLIST_KEY)) || [];
  } catch (_) {
    return [];
  }
}

function toggleWatchlist(movie) {
  const list = getWatchlist();
  const idx = list.findIndex(m => m.id === movie.id);
  if (idx >= 0) list.splice(idx, 1);
  else list.unshift({ id: movie.id, title: movie.title });
  localStorage.setItem(WATCHLIST_KEY, JSON.stringify(list));
}
```

The history code follows the same local-storage design and caps the retained list at 25 entries. The Drizzle schema in the archive defines a user table only; it does not define watchlist or watch-history tables. The Supabase PostgreSQL schema belongs to the separate API-response cache service.

5.5 Responsive User Experience

The static cinema page includes a fixed desktop sidebar and a mobile top bar with a navigation drawer. The design uses glass panels, neon accent colors, skeleton loading states, toast notifications, empty states, and explicit error states. The HTML uses semantic labels and live regions for several status messages, and the React shell prevents recursive rendering when the application is embedded in its own iframe.

6 Detailed Architecture

6.1 React Authentication Shell

The React router exposes the home page and a not-found page. The home page listens to Firebase's `onAuthStateChanged` event. When the current user is authenticated and the email is verified, the page renders `CinemaDashboard`; otherwise it renders the authentication or verification experience. The dashboard is loaded in an iframe from `/cinema/app.html`, separating the cinema interface from the React authentication shell while retaining same-origin communication.

The parent page listens for a `CINEMA_AUTH_LOGOUT` message from the iframe. This allows the embedded dashboard to request a global Firebase logout without duplicating authentication logic. The design keeps Firebase ownership in the parent application and makes the cinema page a post-login experience.

The central routing decision is implemented in the React home page. The following excerpt shows that a verified Firebase user receives the cinema dashboard, while unauthenticated or unverified users remain in the authentication flow.

Listing 2: Authentication-controlled rendering from `client/src/pages/Home.tsx`.

```
if (user && user.emailVerified && !showVerification) {
  return <CinemaDashboard user={user} />;
}

if (showVerification) {
  return <main aria-label="Email verification required">
    {/* verification interface */}
  </main>;
}
```

6.2 Node/Express Layer

The project includes a Node/Express server with tRPC support and a set of core helpers for context, cookies, environment variables, OAuth, storage, notifications, map services, image generation, and voice transcription. The package scripts support development with

a TypeScript watcher, production bundling with Vite and esbuild, static type checking, testing, formatting, and database generation/migration commands.

The server entry point creates an Express application, registers the storage and OAuth routes, registers the cinema routes, and mounts the tRPC adapter at `/api/trpc`. This confirms that Express and tRPC are active server components rather than packages listed only as unused dependencies.

Listing 3: Express and tRPC registration from `server/_core/index.ts`.

```
const app = express();
const server = createServer(app);

registerStorageProxy(app);
registerOAuthRoutes(app);
registerCinemaRoutes(app);

app.use(
  "/api/trpc",
  createExpressMiddleware({
    router: appRouter,
    createContext,
  })
);
```

The cinema routes file also contains a direct TMDB/YouTube implementation. It uses a process-local JavaScript Map with a ten-minute cache lifetime. Consequently, the Node route and the separate FastAPI service should be treated as two available server-side implementations, not as a single automatically proven request path.

Listing 4: Direct in-memory cache used by the Node cinema routes.

```
const cache = new Map<string, { expires: number; value: unknown }>();
const CACHE_TTL = 10 * 60 * 1000;

function cacheGet<T>(key: string) {
  const item = cache.get(key);
  return item && item.expires > Date.now()
    ? item.value as T
    : null;
}
```

6.3 FastAPI Cache Service

The Python service is an independent FastAPI application. It exposes a health endpoint, search endpoint, movie-list endpoints, movie-details endpoint, full movie-analysis endpoint, and analysis-only endpoint. The service uses asynchronous SQLAlchemy sessions and asynchronous HTTP requests, which allows external API calls and database operations to be awaited without blocking the event loop. It is best described as a separate cache-service implementation or deployment option in the submitted archive. The Node file `server/cinemaRoutes.ts` also contains a direct TMDB/YouTube implementation with a

short in-memory cache, so the source does not prove that every runtime request is routed through FastAPI.

The cache record contains a stable key, endpoint category, JSON payload, expiration timestamp, and hit count. A cache hit returns the stored payload and increments the hit count. A miss or expired record invokes the relevant fetcher, stores the fresh payload, resets the expiration timestamp, and returns the result.

The database-backed cache behavior is implemented by the following function. It first attempts a database lookup. If a valid row exists, the hit counter is incremented. Otherwise, the fetcher is called and the result is persisted.

Listing 5: Persistent cache control from `python-cache-service/app/cache.py`.

```
async def cached(session, key, endpoint, ttl_seconds, fetcher):
    existing = await cache_get(session, key)
    if existing is not None:
        return existing
    fresh = await fetcher()
    return await cache_set(
        session, key, endpoint, fresh, ttl_seconds
    )
```

Table 2: Cache policy implemented by the Python service.

Data category	Base TTL	Reasoning
Popular, now playing, upcoming, top rated	6 hours	Lists change regularly but do not require a request for every visitor.
Search results	12 hours	Search results are relatively stable and can be reused.
Movie details	30 days	Released-movie metadata changes infrequently.
YouTube sentiment analysis	3 days	Review discussions can grow and should refresh more often than static metadata.

6.4 Dynamic Expiration

The service adjusts TTL according to popularity and release age. Highly popular or newly released movies receive shorter effective TTLs, while older and less popular movies may remain cached longer. The implementation enforces a minimum TTL of 30 minutes and a maximum TTL of 90 days. Very new titles have special handling: titles released within two days bypass the analysis cache, while titles released within seven days use a one-hour analysis TTL.

6.4.1 Expiry and Fetch Formula

For a response stored at time t_0 , the expiry time is calculated as

$$t_{\text{expiry}} = t_0 + I,$$

where I is the selected refresh interval. The cached response is reused while

$$t_{\text{now}} < t_{\text{expiry}}.$$

When the cache is missing or $t_{\text{now}} \geq t_{\text{expiry}}$, the application starts a new TMDB or YouTube request, stores the response, and starts the interval again.

Table 3: Data-fetch intervals used by the persistent cache service.

Data	Refresh interval	Fetch behavior
Popular, now playing, upcoming, and top-rated lists	6 hours	TMDB is called again after six hours or when no cached list exists.
Search results	12 hours	TMDB is called again after twelve hours or when the search key is missing.
Movie details	30 days	TMDB is called again after thirty days because metadata changes less frequently.
Highly popular or newly released movies	Shorter than the normal interval when the dynamic policy applies	Frequently changing titles are refreshed sooner than older, less popular titles.
Older or less popular movies	Longer than the normal interval when the dynamic policy applies	Stable titles can remain cached longer; the implementation limits the interval to its configured bounds.
YouTube review sentiment	3 days normally	YouTube is called again after three days when the review-analysis cache expires.
YouTube reviews for movies released within 2 days	No cache bypass	Fresh YouTube reviews are fetched for every analysis request.
YouTube reviews for movies released within 3–7 days	1 hour	Recent titles use a short one-hour review interval.

The Node/Express cinema-route implementation is separate from the persistent FastAPI cache and uses a fixed ten-minute in-memory interval:

$$I_{\text{Node}} = 10 \text{ minutes}, \quad t_{\text{expiry}} = \text{Date.now}() + I_{\text{Node}}.$$

The table therefore describes the FastAPI/Supabase cache policy, while the ten-minute expression describes the direct Node route.

7 Data Flow

A typical movie-analysis request follows these steps:

1. The authenticated user selects a movie in the cinema dashboard.
2. The client requests the movie's full data through the application API.
3. The Node/Express cinema route handles the request in the current application path. If the FastAPI cache service is configured and used as the deployment target, the request instead follows the persistent-cache path described below.
4. In the persistent-cache path, the cache service checks for a details key such as `movie:27205:details`; in the direct Node path, the server checks its short-lived in-memory cache.
5. On a miss, the relevant TMDB client requests movie data with credits and videos appended.
6. The service normalizes the response into title, dates, overview, images, rating, director, cast, genres, and trailer fields.
7. The relevant server path checks its review-analysis cache and, when necessary, searches YouTube for review videos and comments.
8. The analysis module splits comments into sentences, cleans tokens, scores positive and negative words, and associates sentences with aspects such as Acting, Story, Visuals, Music, and Pacing.
9. The service returns movie metadata, sentiment statistics, pros, cons, representative positive and negative sentences, and any source errors.
10. The client renders the detail view and records the analysis in local watch history.

8 Sentiment-Analysis Method

The analysis implementation is a deterministic lexicon-based classifier. It defines positive terms such as *excellent*, *funny*, and *masterpiece*, and negative terms such as *boring*, *weak*, and *terrible*. Each sentence receives a score equal to the number of positive matches minus the number of negative matches. Neutral sentences with a score of zero are excluded from the scored total.

For each sentence, the system detects aspect keywords. For example, words related to actors and performances map to the Acting aspect, while words related to plot and writing map to Story. Aspect buckets retain positive and negative mention counts and up to three examples. The overall positive percentage is calculated as

$$P = 100 \times \frac{N_+}{N_+ + N_-},$$

where N_+ and N_- are the counts of positive and negative scored sentences. The verdict thresholds are:

- $P \geq 65\%$: Mostly Positive;

- $P \leq 35\%$: Mostly Negative;
- otherwise: Mixed; and
- no scored sentences: Not enough data.

This method is transparent and inexpensive, but it is not a full natural-language understanding model. It may miss negation, sarcasm, context, spelling variation, and domain-specific meanings. The report therefore treats the sentiment result as an indicative summary rather than a definitive measure of audience opinion.

The classification thresholds are visible in the implementation. The following excerpt shows the final verdict calculation used by the Node cinema route; the Python service contains the same threshold logic.

Listing 6: Sentiment verdict calculation from `server/cinemaRoutes.ts`.

```
const total = positive + negative;
const pct = total
  ? Math.round((positive / total) * 1000) / 10
  : null;
const verdict = pct == null
  ? "Not enough data"
  : pct >= 65
    ? "Mostly Positive"
  : pct <= 35
    ? "Mostly Negative"
  : "Mixed";
```

9 Database and Persistence Design

The project uses two persistence strategies for different purposes. Shared movie/API results are stored in Supabase PostgreSQL so that all users benefit from the same cache. Personal interaction state is stored locally in each browser because watchlist and history are lightweight, user-specific preferences in the current prototype.

Table 4: Persistence responsibilities.

Data	Storage	Purpose
Cached movie/search/review responses	PostgreSQL	Shared reuse, TTL enforcement, hit-count measurement
Watchlist	Browser localStorage	Fast, client-specific saved titles
Watch history	Browser localStorage	Recent analysis history, capped at 25 items
Authentication profile	Firebase Authentication and related profile bridge	Account identity and login lifecycle

10 Security, Reliability, and Error Handling

The implementation includes several reliability and security-oriented behaviors. Firebase owns credential handling, email verification, password reset, and provider reauthentication. The client validates email format and prevents submission while an authentication request is already in progress. The cache service refuses to run as ready when required TMDB or YouTube keys are missing, and its health endpoint reports the missing key names.

The TMDB client tries two API base hosts and raises a controlled error if both fail. The YouTube client skips unavailable comment threads instead of failing the entire review collection. API route handlers convert upstream failures into HTTP 502 responses with simplified error messages. The browser presents retry controls for network failures and preserves user feedback through toast or status regions.

The source archive contains environment files and operational scripts. In a production deployment, secrets should be provided through a secret manager or deployment environment rather than committed to source control. CORS origins, service URLs, API keys, database URLs, and Firebase configuration should be reviewed before release.

11 Testing and Build Strategy

The root package provides the following engineering commands:

```
pnpm dev      # development server with TypeScript watcher
pnpm build    # Vite frontend build and esbuild server bundle
pnpm check    # TypeScript type checking
pnpm test     # Vitest test suite
pnpm format   # Prettier formatting
pnpm db:push  # Drizzle generation and migration command
```

The source includes authentication-related tests, API-key tests, Vitest configuration, TypeScript configuration, and a Vite configuration. The Python service includes separate run scripts for Windows and Unix-like systems. Its setup documentation describes automatic virtual-environment creation, dependency installation, table creation, health verification, and cache-hit verification through the Supabase table editor.

12 Evaluation

The project demonstrates a coherent prototype architecture rather than a single-page mockup. Its strongest qualities are the separation between authentication and the cinema experience, the use of persistent shared caching, the explicit cache policy, and the end-to-end integration from movie discovery to sentiment analysis. The dashboard also includes practical interaction details such as loading placeholders, retry handling, responsive navigation, watchlist badges, and local history.

The main limitations are equally clear. The sentiment classifier is lexicon-based and therefore limited in linguistic coverage. Watchlist and history are browser-local rather than synchronized across devices. The cache service uses a simple `create_all` approach according to its documentation, so future schema evolution would benefit from Alembic or a migration strategy. The prototype also depends on external services and API keys, which introduces quota, latency, availability, and terms-of-use considerations.

13 Future Enhancements

Potential improvements include:

1. replace the lexicon classifier with a validated transformer or hosted sentiment model while retaining the current cache interface;
2. add user-scoped server persistence for watchlists and history so that preferences follow the user across devices;
3. introduce database migrations, cache eviction/observability dashboards, and metrics for hit ratio and upstream latency;
4. add automated integration tests covering cache hit, cache miss, expiry, external failure, and recently released movie behavior;
5. improve language support, negation handling, sarcasm detection, and review-source quality filtering;
6. add role-based administration for managing API configuration and service health; and
7. package the Node and FastAPI services with a reproducible deployment configuration.

14 Conclusion

Vibecheck Cinema is a full-stack movie discovery and media-intelligence prototype that combines a responsive static cinema dashboard with a React authentication shell and two server-side data paths. Firebase provides the account lifecycle. React and TypeScript provide the authenticated application shell, while the main cinema interface is implemented in HTML, CSS, and JavaScript. Node.js/Express serves the application and exposes the active cinema routes, including direct TMDB/YouTube access with an in-memory cache. The archive also includes a FastAPI service that normalizes TMDB data, collects YouTube review comments, produces an interpretable sentiment summary, and caches results in PostgreSQL according to dynamic expiration policies.

The architecture is suitable for a prototype or academic demonstration because each major responsibility is visible and independently testable. Future production work should focus on stronger language analysis, cross-device persistence, schema migration, observability, automated integration tests, and secure deployment of external-service credentials.

A Representative API Endpoints

Endpoint	Purpose
GET /api/health	Reports service status and whether required API keys are configured.

Endpoint	Purpose
GET /api/search?q=...	Returns cached or freshly fetched movie search results.
GET /api/movies/popular	Returns the popular movie list.
GET /api/movies/now-playing	Returns movies currently in theatres.
GET /api/movies/upcoming	Returns upcoming movie releases.
GET /api/movies/top-rated	Returns top-rated movie titles.
GET /api/movies/{movie_id}	Returns normalized movie details.
GET /api/movies/{movie_id}/analysis	Returns the sentiment-analysis result.
GET /api/movies/{movie_id}/full	Returns movie details and analysis together.

B Source-Based Notes

This report was prepared by inspecting the submitted source archive. The documented behavior reflects the implementation found in the archive, including the React authentication flow, iframe-based cinema dashboard, JavaScript localStorage features, FastAPI routes, TMDb and YouTube clients, cache policies, and lexicon-based analysis module. Runtime performance and external API results were not independently benchmarked because they depend on deployment configuration and credentials.